INSTITUTO TECNOLOGICO JOSÉ MARIO MOLINA PASQUEL Y HENRIQUEZ

UNIDAD ACADEMICA ZAPOTLANEJO

Manual de programación

Alumno: Oscar Alejandro Alvarez Aceves

Carrera: Ingeniería en informática

Materia: Administración de datos

Docente: Osvaldo Rene Rojo Roa

Parcial 2

Unidad 2

Programa 1 Creación y lectura de archivo

El programa creara un archivo donde pide por medio de consola los datos de 6 personas y guardándolo en un archivo .txt , si el archivo ya existe mostrara los datos guardados en el

```
using static System.Runtime.InteropServices.JavaScript.JSType;
class program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hola mundo");
        // Print greeting message / Imprime mensaje de saludo

        string nombrearchivo = "Datos.txt";
        // File name variable / Variable con el nombre del archivo
        string nombre;
        // Variable to store name / Variable para guardar el nombre
        int edad;
        // Variable to store age / Variable para guardar la edad
        double estatura;
        // Variable to store height / Variable para guardar la estatura
        string datos;
        // Variable to store each line read from file / Variable para guardar cada línea leída del archivo
        StreamReader leerarchivo = null;
        // Reader object initialized as null / Objeto lector inicializado en null
        StreamWriter archivo = null;
        // Writer object initialized as null / Objeto escritor inicializado en null
        if (File.Exists(nombrearchivo))
        // Check if file exists / Verifica si el archivo existe
        {
            leerarchivo = File.OpenText(nombrearchivo);
            // Open file for reading / Abre el archivo para lectura
            do
            {
                datos = leerarchivo.ReadLine();
                // Read one line from file / Lee una línea del archivo
                string[] d = datos.Split(" _ ");
                // Split line using separator / Divide la línea usando el separador
                Console.WriteLine("El nombre es: " + d[0]);
                // Print name (first position) / Imprime el nombre (posición 0)
                Console.WriteLine("la edad es: " + d[1]);
                // Print age (second position) / Imprime la edad (posición 1)
                Console.WriteLine("la estatura es: " + d[2]);
                // Print height (third position) / Imprime la estatura (posición 2)
            } while (datos != null);
            // Repeat until line is null / Repite hasta que la línea sea null
            leerarchivo.Close();
            // Close reader / Cierra el lector
        }
        else
        {
            archivo = File.AppendText(nombrearchivo);
            // Create/open file for writing / Crea o abre el archivo para escritura
            for (int i = 1; i < 6; i++)
            // Loop 5 times / Ciclo que se ejecuta 5 veces
            {
                Console.WriteLine("Escribe el nombre");
                // Ask for name / Solicita el nombre
                nombre = Console.ReadLine();
                // Read name from console / Lee el nombre desde consola
                Console.WriteLine("Escribe la edad");
                // Ask for age / Solicita la edad
                edad = Convert.ToInt32(Console.ReadLine());
                // Convert input to int / Convierte la entrada a entero
                Console.WriteLine("Escribe la estatura");
                // Ask for height / Solicita la estatura
                estatura = Convert.ToDouble(Console.ReadLine());
                // Convert input to double / Convierte la entrada a decimal (double)
                archivo.WriteLine(nombre + "_" + edad + "_" + estatura);
                // Write data to file separated by "_" / Escribe los datos en el archivo separados por "_"
                i++;
                // Manually increment i (extra increment) / Incrementa manualmente i (incremento extra)
            }
            archivo.Close();
            // Close writer / Cierra el escritor
        }
    }
}
```

Ilustración 1 Código programa 1

```
Hola mundo
El nombre es: Oscar
la edad es: 23
la estatura es: 1.72
El nombre es: diego
la edad es: 20
la estatura es: 1.75
El nombre es: luis
la edad es: 21
la estatura es: 1.8
El nombre es: armando
la edad es: 23
la estatura es: 1.75
El nombre es: maria
la edad es: 22
la estatura es: 1.67
```

Ilustración 2 Ejecución programa 1

Programa 2 Menú, creación, eliminación y modificación de archivos con validación

El programa iniciaría con un menú de opciones donde se mostrarán las diferentes acciones que el usuario puede utilizar, si ingresa una opción no valida el menú se repetirá de manera infinita hasta seleccionar una opción o terminar su ejecución con la opción salir.

- La primera opción agregara información de personas validada a un archivo .txt
- La segunda opción eliminara el archivo existente
- La tercera opción mostrara la información del archivo y solicita ingresar el nombre de la persona que desea eliminar del archivo, para después crear un archivo temporal sin los datos de la persona seleccionada y remplazando al original.
- La cuarta opción mostrara la información dentro del archivo
- La quinta opción es para salir del programa

```
0 referencias
class Program
{

    // Instance of the validation class
    // Instancia de la clase de validaciones
    static Validaciones val = new Validaciones();

    0 referencias
    static void Main(string[] args)
    {
        // Variable that stores the menu option selected by the user
        // Variable que guarda la opción seleccionada del menú
        int op = 0;

        // Loop that keeps the program running until the user selects option 5
        // Ciclo que mantiene el programa ejecutándose hasta que el usuario elija la opción 5
        do
        {

            op = Menus();
            switch (op)
            {
                case 1:
                    // Call method to add a new record to the file
                    // Llama al método para agregar un nuevo registro al archivo
                    Agregar();
                    break;
                case 2:
                    // Call method to delete the entire file
                    // Llama al método para eliminar todo el archivo
```

Ilustración 3 Código programa 2

```
                    Eliminarachivo();
                    Console.WriteLine("Eliminar");
                    break;
                case 3:
                    // Call method to delete a specific element from the file
                    // Llama al método para eliminar un elemento específico del archivo
                    EliminarElementoArchivo();
                    Console.WriteLine("Eliminar dato");
                    break;
                case 4:
                    // Call method to display file content
                    // Llama al método para mostrar el contenido del archivo
                    Mostrar();
                    break;
                case 5:
                    // Exit option
                    // Opción para salir del programa
                    break;
            }
        } while (op != 5);

    }

    1 referencia
    static void EliminarElementoArchivo()
    {
        // Variable to store the name to delete
        // Variable para guardar el nombre que se desea eliminar
        string nombre;

        // Show current data before deleting
```

Ilustración 4 Código programa 2

```
56          // Variable para guardar el nombre que se desea eliminar
57          string nombre;
58
59          // Show current data before deleting
60          // Muestra los datos actuales antes de eliminar
61          Mostrar();
62
63          Console.WriteLine("Escribe el nombre a eliminar");
64          nombre = Console.ReadLine();
65
66          // Reader object to read the file
67          // Objeto lector para leer el archivo
68          StreamReader leerArchivo = null;
69
70          // Variable that stores each line read
71          // Variable que guarda cada línea leída
72          string datos = null;
73
74          // Check if file exists
75          // Verifica si el archivo existe
76          if (File.Exists("Datos.txt"))
77          {
78              // Open file for reading
79              // Abre el archivo para lectura
80              leerArchivo = File.OpenText("Datos.txt");
81
82              do
83              {
84                  // Read a line from the file
85                  // Lee una línea del archivo
```

Ilustración 5 Código programa 2

```
86                  datos = leerArchivo.ReadLine();
87
88                  if (datos != null)
89                  {
90                      // Split the line into parts (name, age, height)
91                      // Divide la línea en partes (nombre, edad, estatura)
92                      string[] d = datos.Split(" _ ");
93
94                      // If the name matches the one to delete
95                      // Si el nombre coincide con el que se quiere eliminar
96                      if (nombre.Equals(d[0]))
97                      {
98                          Console.WriteLine("Si lo encontro");
99                      }
100                     else
101                     {
102                         // If it does not match, save the line into an auxiliary file
103                         // Si no coincide, guarda la línea en un archivo auxiliar
104                         StreamWriter archivo = null;
105                         archivo = File.AppendText("DatosAux.txt");
106                         archivo.WriteLine(datos);
107                         archivo.Close();
108                     }
109                 }
110             } while (datos != null);
111
112             // Close the reader
113             // Cierra el lector
114             leerArchivo.Close();
115
```

Ilustración 6 Código programa 2

```
            // Delete original file
            // Elimina el archivo original
            File.Delete("Datos.txt");

            // Copy auxiliary file to replace the original
            // Copia el archivo auxiliar para reemplazar el original
            File.Copy("DatosAux.txt", "Datos.txt");

            // Delete auxiliary file
            // Elimina el archivo auxiliar
            if (File.Exists("DatosAux.txt"))
            {
                File.Delete("DatosAux.txt");
            }
        }
    }

    static void Eliminarachivo()
    {

        // Check if the file exists
        // Verifica si el archivo existe
        if (File.Exists("Datos.txt"))
        {
            // Delete the file
            // Elimina el archivo
            File.Delete("Datos.txt");
        }
        else
        {
```

Ilustración 7 Código programa 2

```
            Console.WriteLine("El archivo no existe");
        }

    }

    static void Agregar()
    {
        // Variables to store user input
        // Variables para guardar datos del usuario
        string nombre, edad, estatura;

        // Writer object for the file
        // Objeto escritor del archivo
        StreamWriter archivo = null;

        // Loop until a valid name is entered
        // Ciclo hasta que se ingrese un nombre válido
        while (true)
        {
            Console.WriteLine("Escribe un nombre");
            nombre = Console.ReadLine();

            // Validate the name
            // Validar el nombre
            if (val.Validarnombre(nombre))
            {
                break;

            }
```

Ilustración 8 Código programa 2

```
            else
            {
                Console.WriteLine("Error al introducir el nombre");
            }
        }

        Console.WriteLine("El nombre correcto es: " + nombre);

        // Loop until a valid age is entered
        // Ciclo hasta que se ingrese una edad válida
        while (true)
        {
            Console.WriteLine("Escribe la edad");
            edad = Console.ReadLine();

            if (val.Validaredad(edad))
            {
                break;

            }
            else
            {
                Console.WriteLine("Error al introducir la edad");
            }
        }

        Console.WriteLine("La edad correcta es: " + edad);

        // Loop until a valid height is entered
        // Ciclo hasta que se ingrese una estatura válida
```

Ilustración 9 Código programa 2

```
        while (true)
        {
            Console.WriteLine("Escribe la estatura");
            estatura = Console.ReadLine();

            if (val.Validarestatura(estatura))
            {
                break;

            }
            else
            {
                Console.WriteLine("Error al introducir la estatura");
            }
        }

        Console.WriteLine("La estatura correcta es: " + estatura);

        // Open file to append data
        // Abre el archivo para agregar datos
        archivo = File.AppendText("Datos.txt");

        // Save data separated by "_"
        // Guarda los datos separados por "_"
        archivo.WriteLine(nombre + " _ " + edad + " _ " + estatura);

        // Close the file
        // Cierra el archivo
        archivo.Close();
    }
```

Ilustración 10 Código programa 2

```
2 referencias
static void Mostrar()
{
    // Reader object
    // Objeto lector
    StreamReader leerarchivo = null;

    // Variable to store lines
    // Variable para guardar líneas
    string datos = null;

    // Check if file exists
    // Verifica si el archivo existe
    if (File.Exists("Datos.txt"))
    {
        leerarchivo = File.OpenText("Datos.txt");

        do
        {
            // Read line
            // Lee línea
            datos = leerarchivo.ReadLine();

            if (datos != null)
            {

                // Split the line into data fields
                // Divide la línea en campos de datos
                string[] d = datos.Split(" _ ");
```

Ilustración 11 Código programa 2

```
                // Display stored data
                // Muestra los datos almacenados
                Console.WriteLine("El nombre es: " + d[0]);
                Console.WriteLine("la edad es: " + d[1]);
                Console.WriteLine("la estatura es: " + d[2]);
            }

        } while (datos != null);

        // Close file reader
        // Cierra el lector del archivo
        leerarchivo.Close();
    }
}

1 referencia
static int Menus()
{
// Label used for menu validation loop
// Etiqueta usada para repetir el menú si hay error
inicio:

    int opcion = 0;

    // Display menu options
    // Muestra las opciones del menú
    Console.WriteLine("*************************************");
    Console.WriteLine("1.- Agregar elementos al archivo");
    Console.WriteLine("2.- Eliminar archivo");
    Console.WriteLine("3.- Eliminar datos en el archivo");
```

Ilustración 12 Código programa 2

```
        Console.WriteLine("*************************************");

        Console.WriteLine("Escribe la opcion del menu");

        // Read user input
        // Lee la opción ingresada
        string op = Console.ReadLine();

        // Validate if the input is a valid number
        // Valida si la entrada es un número válido
        if (val.Validaredad(op))
        {
            opcion = int.Parse(op);
        }
        else
        {
            Console.WriteLine("Error en el menu");
            goto inicio; // Return to menu if validation fails
        }

        return opcion;
    }
}
```

Ilustración 13 Código programa 2

```
class Validaciones
{
    1 referencia
    public bool Validarnombre(string n) {
        int c = 0;
        byte[] ascii =Encoding.ASCII.GetBytes(n);
        foreach (byte b in ascii)
        {
            if ((b >97 && b <122 )||(b >65&&b <90)|| b ==32)
            {
                c ++;
            }
        }
        if (c == n.Length)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    2 referencias
    public bool Validaredad(string e) {
        bool r = (Regex.IsMatch(e, @"^[0-9]+$"));
        return r;
    }
    1 referencia
    public bool Validarestatura(string e) {
        bool r = false;
        try {
            float estatura = float.Parse(e);
            r = true;
        } catch
        {
            r = false;
        }
        return r;
    }
```

Ilustración 14 Validación programa 2

```
**********************************
1.- Agregar elementos al archivo
2.- Eliminar archivo
3.- Eliminar datos en el archivo
4.- Mostrar contenido del archivo
5.- Salir
**********************************
Escribe la opcion del menu
4
El nombre es: oscar
la edad es: 23
la estatura es: 172
El nombre es: diego
la edad es: 21
la estatura es: 189
El nombre es: carlos
la edad es: 21
la estatura es: 189
**********************************
1.- Agregar elementos al archivo
2.- Eliminar archivo
3.- Eliminar datos en el archivo
4.- Mostrar contenido del archivo
5.- Salir
**********************************
Escribe la opcion del menu
```

Ilustración 15 ejecución Código 2

Programa 3 ciclos infinitos con creación de archivos con datos validados

El programa iniciaría solicitando datos y validando los caracteres ingresados, si no cumple con el formato deseado mostrara error y repetirá la acción hasta que sea correcta, al final preguntara si desea agregar otro dato, mientras la opción sea Y o y se repetirá el programa hasta que el usuario conteste lo contrario

```
using static System.Runtime.InteropServices.JavaScript.JSType;

class Program
{
    static Validaciones val = new Validaciones();
    // Create instance of validation class / Crea una instancia de la clase de validaciones

    static void Main(string[] args)
    {
        StreamReader leerarchivo = null;
        // Reader object initialized as null / Objeto lector inicializado en null

        StreamWriter archivo = null;
        // Writer object initialized as null / Objeto escritor inicializado en null

        string respuesta = null;
        // Variable to store user response / Variable para guardar la respuesta del usuario

        int op = 0;
        // Integer variable (not used) / Variable entera (no utilizada)

        archivo = File.AppendText("Datos.txt");
        // Open file in append mode / Abre el archivo en modo agregar

        do
        {
        inicio:
            // Label for goto / Etiqueta para usar con goto

            string nombre;
            // Variable for name / Variable para el nombre

            string edad;
            // Variable for age / Variable para la edad

            string salario;
            // Variable for salary / Variable para el salario

            string correo;
            // Variable for email / Variable para el correo

            string numero;
            // Variable for phone number / Variable para el número telefónico

            string contraseña;
            // Variable for password / Variable para la contraseña

            string aux;
            // Auxiliary variable / Variable auxiliar

            Console.WriteLine("Ingresa un nombre");
            // Ask for name / Solicita el nombre
```

Ilustración 16 Código programa 3

```
            Console.WriteLine("Ingresa un nombre");
            // Ask for name / Solicita el nombre

            nombre = Console.ReadLine();
            // Read name from console / Lee el nombre desde consola

            if (!val.ValidarNombre(nombre))
            // Validate name / Valida el nombre
            {
                Console.WriteLine("Nombre incorrecto");
                // Show error if invalid / Muestra error si es inválido
            }
            else
            {
                Console.WriteLine("ingresa la edad");
                // Ask for age / Solicita la edad

                edad = Console.ReadLine();
                // Read age / Lee la edad

                if (!val.ValidarEdad(edad))
                // Validate age / Valida la edad
                {
                    Console.WriteLine("Edad incorrecta");
                    // Show error if invalid / Muestra error si es inválida
                }
                else
                {
                    Console.WriteLine("Ingresa el salario");
                    // Ask for salary / Solicita el salario

                    salario = Console.ReadLine();
                    // Read salary / Lee el salario

                    if (!val.ValidarSalario(salario))
                    // Validate salary / Valida el salario
                    {
                        Console.WriteLine("Salario incorrecta");
                        // Show error if invalid / Muestra error si es inválido
                    }
                    else
                    {
                        Console.WriteLine("Ingresa el correo electronico");
                        // Ask for email / Solicita el correo

                        correo = Console.ReadLine();
                        // Read email / Lee el correo

                        if (!val.ValidarCorreo(correo))
                        // Validate email / Valida el correo
                        {
                            Console.WriteLine("correo incorrecto");
                            // Show error if invalid / Muestra error si es inválido
```

Ilustración 17 Código programa 3

```
                    Console.WriteLine("correo incorrecto");
                    // Show error if invalid / Muestra error si es inválido
                }
                else
                {
                    Console.WriteLine("Ingresa el numero de telefono");
                    // Ask for phone number / Solicita el número telefónico

                    numero = Console.ReadLine();
                    // Read phone number / Lee el número

                    if (!val.ValidarNumero(numero))
                    // Validate phone number / Valida el número
                    {
                        Console.WriteLine("Numero incorrecto");
                        // Show error if invalid / Muestra error si es inválido
                    }
                    else
                    {
                        Console.WriteLine("Ingresa la contraseña");
                        // Ask for password / Solicita la contraseña

                        contraseña = Console.ReadLine();
                        // Read password / Lee la contraseña

                        if (!val.ValidarContraseña(contraseña))
                        // Validate password / Valida la contraseña
                        {
                            Console.WriteLine("contraseña incorrecta");
                            // Show error if invalid / Muestra error si es inválida
                        }
                        else
                        {
                            Console.WriteLine("El nombre es: " + nombre.ToUpper());
                            // Print name in uppercase / Imprime el nombre en mayúsculas

                            Console.WriteLine("La edad es: " + edad);
                            // Print age / Imprime la edad

                            Console.WriteLine("El correo es: " + correo.ToLower());
                            // Print email in lowercase / Imprime el correo en minúsculas

                            Console.WriteLine("El salario es: " + salario);
                            // Print salary / Imprime el salario

                            aux = "(" + numero.Substring(0, 2) + ") " + numero.Substring(2, 4) + "-" + numero.Substring(6, 4);
                            // Format phone number / Da formato al número telefónico

                            Console.WriteLine("El numero es: " + aux);
                            // Print formatted phone number / Imprime el número formateado

                            Console.WriteLine("La longitud de la contraseña es: " + contraseña.Length);
                            // Print password length / Imprime la longitud de la contraseña
```

Ilustración 18 Código programa 3

```
                            // Format phone number / Da formato al número telefónico

                            Console.WriteLine("El numero es: " + aux);
                            // Print formatted phone number / Imprime el número formateado

                            Console.WriteLine("La longitud de la contraseña es: " + contraseña.Length);
                            // Print password length / Imprime la longitud de la contraseña

                            archivo.WriteLine(nombre + "_" + edad + "_" + salario + "_" + correo + "_" + numero + "_" + contraseña);
                            // Write data to file separated by "_" / Escribe los datos en el archivo separados por "_"

                            archivo.Close();
                            // Close file / Cierra el archivo

                            Console.WriteLine("¿Desea ingresar otra persona Y/N?");
                            // Ask if user wants to continue / Pregunta si desea continuar

                            respuesta = Console.ReadLine();
                            // Read response / Lee la respuesta

                            if ((respuesta == "Y") || (respuesta == "y"))
                            // Check if user wants to continue / Verifica si el usuario quiere continuar
                            {
                                goto inicio;
                                // Jump to label inicio / Salta a la etiqueta inicio
                            }
                            else
                            {
                                break;
                                // Exit loop / Sale del ciclo
                            }
                        }
                    }
                }
            }
        }
    } while (true);
    // Infinite loop / Ciclo infinito
}
```

Ilustración 19 Código programa 3

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;


2 referencias
class Validaciones
{
    1 referencia
    public bool ValidarNombre(string N)
    {
        int c = 0;
        // Counter for valid characters / Contador de caracteres válidos

        byte[] ascii = Encoding.ASCII.GetBytes(N);
        // Convert string to ASCII bytes / Convierte la cadena a bytes ASCII

        foreach (byte b in ascii)
        // Iterate each character byte / Recorre cada byte del texto
        {
            if ((b ≥ 97 && b ≤ 122) || (b ≥ 65 && b ≤ 90) || b == 32)
            // Check if lowercase, uppercase or space / Verifica si es minúscula, mayúscula o espacio
            {
                c++;
                // Increase counter if valid / Aumenta el contador si es válido
            }
        }

        if (c == N.Length)
        // If all characters are valid / Si todos los caracteres son válidos
        {
            return true;
            // Name is valid / Nombre válido
        }
        else
        {
            return false;
            // Name is invalid / Nombre inválido
        }
    }

    1 referencia
    public bool ValidarEdad(string E)
    {
        int e = 0;
        // Variable to store converted age / Variable para guardar la edad convertida

        try
        {
            e = Convert.ToInt32(E);
            // Convert string to integer / Convierte la cadena a entero
```

Ilustración 20 Verificación programa 3

```
            e = Convert.ToInt32(E);
            // Convert string to integer / Convierte la cadena a entero

            if ((e ≥ 18 && e ≤ 99))
            // Check if age is between 18 and 99 / Verifica si la edad está entre 18 y 99
            {
                return true;
                // Valid age / Edad válida
            }
            else
            {
                return false;
                // Age out of range / Edad fuera de rango
            }
        }
        catch
        {
            return false;
            // Return false if conversion fails / Retorna falso si falla la conversión
        }
    }

    1 referencia
    public bool ValidarSalario(string S)
    {
        double s;
        // Variable to store salary / Variable para guardar el salario

        try
        {
            s = Convert.ToDouble(S);
            // Convert string to double / Convierte la cadena a decimal

            if (s < 0)
            // Check if salary is negative / Verifica si el salario es negativo
            {
                return false;
                // Invalid salary / Salario inválido
            }
            else
            {
                return true;
                // Valid salary / Salario válido
            }
        }
        catch
        {
            return false;
            // Return false if conversion fails / Retorna falso si falla la conversión
        }
    }

    1 referencia
    public bool ValidarCorreo(string C)
```

Ilustración 21 Verificación programa 3

```
        {
            bool bA = false;
            // Flag for '@' symbol / Bandera para símbolo '@'

            bool bB = false;
            // Flag for '.' symbol / Bandera para símbolo '.'

            byte[] ascii = Encoding.ASCII.GetBytes(C);
            // Convert string to ASCII bytes / Convierte la cadena a bytes ASCII

            foreach (byte b in ascii)
            // Iterate through each character / Recorre cada carácter
            {
                if (b == 64)
                // ASCII 64 = '@' / 64 es '@'
                {
                    bA = true;
                    // Mark that '@' exists / Marca que existe '@'
                }

                if (b == 46)
                // ASCII 46 = '.' / 46 es '.'
                {
                    string[] partes = C.Split('.');
                    // Split email by '.' / Divide el correo por '.'

                    if (partes.Length >= 1)
                    // Check if split generated parts / Verifica que haya partes
                    {
                        bB = true;
                        // Mark that '.' exists / Marca que existe '.'
                    }
                }
            }

            if (bA == true && bB == true)
            // Email must contain both '@' and '.' / Debe contener '@' y '.'
            {
                return true;
                // Valid email / Correo válido
            }
            else
            {
                return false;
                // Invalid email / Correo inválido
            }
        }

        public bool ValidarNumero(string Nu)
        {
            string N = Nu;
            // Copy input string / Copia la cadena recibida

            double aux = 0;
            // Auxiliary numeric variable / Variable auxiliar numérica

            try
            {
                aux = double.Parse(N);
                // Try converting to number / Intenta convertir a número

                if ((N.Length == 10))
                // Check if length is 10 digits / Verifica que tenga 10 dígitos
                {
                    return true;
                    // Valid phone number / Número válido
                }
                else
                {
                    return false;
                    // Invalid length / Longitud inválida
                }
            }
            catch
            {
                return false;
                // Return false if conversion fails / Retorna falso si falla la conversión
            }
        }

        1 referencia
        public bool ValidarContraseña(string Co)
        {
            bool bA = false;
            // Flag for uppercase letter / Bandera para mayúscula

            bool bB = false;
            // Flag (repeated condition) / Bandera (condición repetida)

            bool bC = false;
            // Flag for number / Bandera para número

            byte[] ascii = Encoding.ASCII.GetBytes(Co);
            // Convert password to ASCII bytes / Convierte la contraseña a bytes ASCII

            foreach (byte b in ascii)
            // Iterate each character / Recorre cada carácter
            {
                if ((b >= 65) && (b <= 90))
                // Check if uppercase letter / Verifica si es mayúscula
                {
                    bA = true;
                    // Mark uppercase found / Marca mayúscula encontrada
                }
```

Ilustración 22 Verificación programa 3

Ilustración 23 Código Programa 3

```
        // Convert password to ASCII bytes / Convierte la contraseña a bytes ASCII

        foreach (byte b in ascii)
        // Iterate each character / Recorre cada carácter
        {
            if ((b ≥ 65) && (b ≤ 90))
            // Check if uppercase letter / Verifica si es mayúscula
            {
                bA = true;
                // Mark uppercase found / Marca mayúscula encontrada
            }

            if ((b ≥ 65) && (b ≤ 90))
            // Same condition repeated / Misma condición repetida
            {
                bB = true;
                // Mark flag / Marca bandera
            }

            if ((b ≥ 48) && (b ≤ 57))
            // Check if numeric digit / Verifica si es número
            {
                bC = true;
                // Mark number found / Marca número encontrado
            }
        }

        if (Co.Length ≥ 8 && bA && bB && bC)
        // Password must be at least 8 characters and meet conditions
        // La contraseña debe tener al menos 8 caracteres y cumplir condiciones
        {
            return true;
            // Valid password / Contraseña válida
        }
        else
        {
            return false;
            // Invalid password / Contraseña inválida
        }
    }
```

Ilustración 24 Código Programa 3

```
Ingresa un nombre
oscar
ingresa la edad
23
Ingresa el salario
999
Ingresa el correo electronico
oscar@gmail.com
Ingresa el numero de telefono
3339458804
1
Ingresa la contraseña
The4l3xrule
El nombre es: OSCAR
La edad es: 23
El correo es: oscar@gmail.com
El salario es: 999
El numero es: (33) 3945-8804
La longitud de la contraseña es: 11
¿Desea ingresar otra persona Y/N?
```

Ilustración 25 Ejecución Programa 3

Unidad 3

Programa1 unity

En este programa utilizando creación modificación y guardado de archivos se genera un videojuego en el cual el jugador puede desplazarse horizontalmente mientras enemigos se crean automáticamente con el objetivo que el jugador suma puntos hasta llegar a la meta de 10 puntos para ser dirigido a un nuevo nivel en donde se mostrará un texto con tu información y los puntajes obtenidos

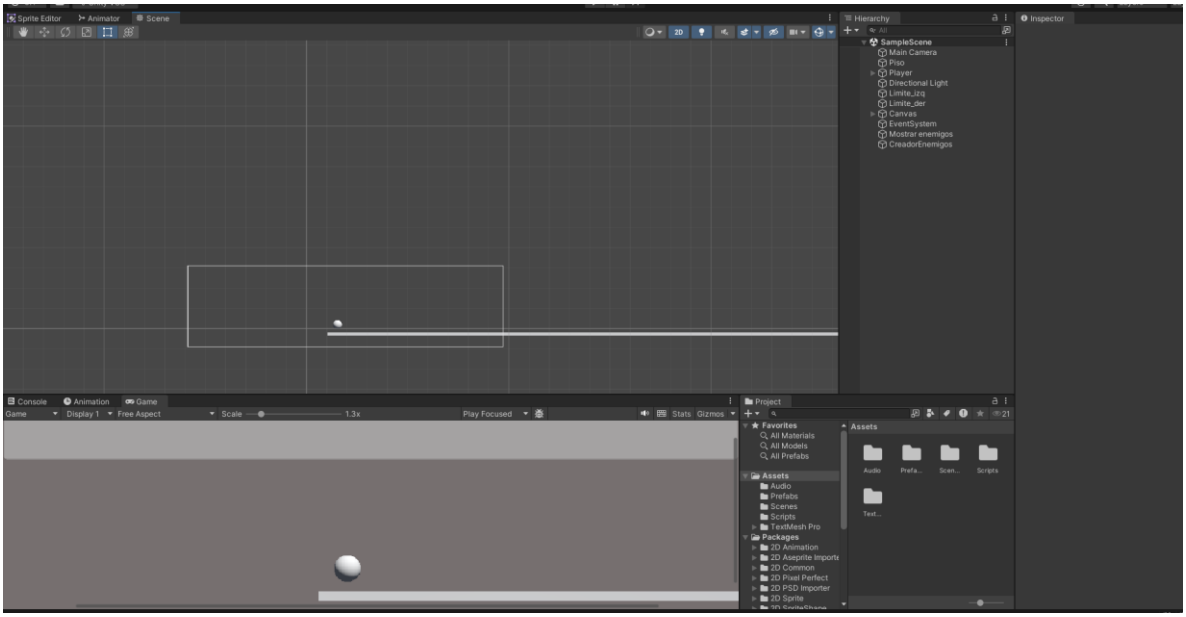

Ilustración 26 Ventana principal

```
using System.Collections.Generic;
using UnityEngine;

Script de Unity (1 referencia de recurso) | 0 referencias
public class Camara : MonoBehaviour
{
public AudioSource musica;
    public Transform target;
    Vector3 posicion;
    public float velocidadCamara = 10.0f;
    Vector3 fueraCamara;
    float yFija;
    public float minX;
    public float minY;
    // Start is called before the first frame update
    Mensaje de Unity | 0 referencias
    void Start()
    {
        fueraCamara = transform.position- target.position; //Buscar al personaje
        yFija = transform.position.y;
        musica.Play();

    }
    Mensaje de Unity | 0 referencias
    private void LateUpdate()
    {
        float objetivo = target.position.x + fueraCamara.x;
        objetivo = Mathf.Clamp(objetivo, minX, minY);
        Vector3 objetivoX = new Vector3(objetivo, yFija, transform.position.z);
        transform.position = Vector3.Lerp(transform.position, objetivoX,velocidadCamara* Time.deltaTime);
    }

    // Update is called once per frame
    //void Update()
    //{
    //    Vector3 nuevaPosicion = target.position + posicion;
    //    transform.position = Vector3.Lerp(transform.position, nuevaPosicion,
    //        velocidadCamara*Time.deltaTime);
    //}
}
```

Ilustración 27 Código Cámara

```
public class Contadordeenemigo : MonoBehaviour
{
    public static int contEnemy = 0;
    public TextMeshProUGUI texto;
    public TextMeshProUGUI n;
    public TextMeshProUGUI men;
    static StreamReader leer;
    string dato = "0";
    public GameObject panel;
    public GameObject panelNivel;
    public TMP_InputField nombre;
    //bool estado = false;
    static StreamWriter archivo;
    private void Awake()
    {
        if (File.Exists("Puntaje.txt"))
        {
            leer = File.OpenText("Puntaje.txt");
            dato = leer.ReadLine();
            leer.Close();
            string[] d = dato.Split("_");
            contEnemy= int.Parse(d[0]);
            nombre.text = d[1];
            texto.text = "Enemigos Eliminados: " + contEnemy;
        }
    }
    public static void EnemigosEliminados() {
        contEnemy++;
    }
     void Update()
    {
        if (Input.GetKeyDown(KeyCode.G))
        {
            print("Guardar datos");
            //panel.SetActive(!estado);
            panel.SetActive(!panel.activeSelf);
            Time.timeScale = 0;
            n.text = "Datos a guardar: "+contEnemy.ToString();
        }
        if (panel.activeSelf == false)
        {
            Time.timeScale = 1;
        }
            texto.text = "Enemigos Eliminados: " + contEnemy.ToString();
        if (contEnemy >= 10)
        {
            Time.timeScale = 0;
```

Ilustración 28 Código contador enemigos

```
            Time.timeScale = 1;
        }
            texto.text = "Enemigos Eliminados: " + contEnemy.ToString();
        if (contEnemy >= 10)
        {
            Time.timeScale = 0;
            panelNivel.SetActive(true);
        }
    }
    public void siguienteEscena()
    {
        guardarAntes();
        SceneManager.LoadScene("Nivel2");
    }
    public void Creararchivo()

    {
        string n = nombre.text;
        if (n.Length == 0)
        {
            men.text = "No tiene nombre";
        }
        else
        {
            if (File.Exists("Puntaje.txt"))
            {
                File.Delete("Puntaje.txt");
            }

            archivo = File.AppendText("Puntaje.txt");
            archivo.WriteLine(contEnemy + "_" + nombre.text);
            archivo.Close();
            men.text = "";
        }
    }
    void guardarAntes()
    {
        if (File.Exists("Puntaje.txt"))
        {
            File.Delete("Puntaje.txt");
        }
        archivo = File.AppendText("Puntaje.txt");
        archivo.WriteLine(contEnemy + "_" + nombre.text);
        archivo.Close();
        men.text = "";
    }
}
```

Ilustración 29 Código Contador enemigos

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Eliminar_Enemigos : MonoBehaviour
{
    // Start is called before the first frame update
    public AudioSource enem;
    void Start()
    {
        enem.Stop();
    }

    // Update is called once per frame
    void Update()
    {

    }
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.tag=="Enemigo")
        {
            print("Eliminar");
            Destroy(collision.transform.root.gameObject);//eliminar padre
            Contadordeenemigo.EnemigosEliminados();
            enem.Play();
        }
    }
}
```

Ilustración 30 Código Eliminar enemigos

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

Script de Unity (1 referencia de recurso) | 0 referencias
public class Enemy : MonoBehaviour
{
    // Start is called before the first frame update
    public float velocidad = 50.0f;
    Mensaje de Unity | 0 referencias
    void Start()
    {

    }

    // Update is called once per frame
    Mensaje de Unity | 0 referencias
    void Update()
    {
        transform.Translate(Vector3.left * velocidad*Time.deltaTime);
    }
    Mensaje de Unity | 0 referencias
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Limite"))
        {
            Destroy(gameObject);
            //anim.SetBool("Saltar",false);
        }
    }

}
```

Ilustración 31 Código movimiento enemigos

```
using System.Collections;
using System.Collections.Generic;
using System.IO;
using TMPro;
using Unity.VisualScripting.Antlr3.Runtime;
using UnityEngine;

Script de Unity (1 referencia de recurso) | 0 referencias
public class MostrarDatos : MonoBehaviour


{
    public static int contEnemy = 0;
    static StreamReader leer;
    static StreamWriter archivo;
    string dato = "0";
    //public TMP_InputField nombre;
    public TextMeshProUGUI texto;
    Mensaje de Unity | 0 referencias
    void Update()
    {
        texto.text = "Enemigos Eliminados: " + contEnemy.ToString();
    }
    Mensaje de Unity | 0 referencias
    private void Awake()
    {
        if (File.Exists("Puntaje.txt"))
        {
            leer = File.OpenText("Puntaje.txt");

            dato = leer.ReadLine();

            leer.Close();
            string[] d = dato.Split("_");
            contEnemy = int.Parse(d[0]);
            texto.text = "Enemigos Eliminados: " + contEnemy;

        }
    }
}
```

Ilustración 32 Código cargar datos

```
public class Mover : MonoBehaviour
{
    public float veloDisp = 20.0f;
    public float velocidad = 100.0f;    //Definir velocidad del jugador
    public float fuerza = 40.0f;        //definir salto del jugador
    public bool tocarPiso=false;        //pariable para el piso
    float posX = 0;
    Rigidbody2D body;
    Mensaje de Unity | 0 referencias
    void Start()
    {
        body = GetComponent<Rigidbody2D>();

    Mensaje de Unity | 0 referencias
    private void FixedUpdate()
    {
        //body.velocity = new Vector2(posX*velocidad*Time.deltaTime, 0);
        body.velocity = new Vector2(posX*velocidad,body.velocity.y);
        if (Input.GetKeyDown(KeyCode.Space) && tocarPiso == true)
        {
            body.AddForce(Vector2.up * fuerza, ForceMode2D.Impulse);
            //anim.SetBool("Saltar",true);
            tocarPiso = false;
        }
        if (body.velocity.y < 0)
        {
            body.gravityScale = 10;
        }
        else
        {
            body.gravityScale = 8;
        }
    }
    Mensaje de Unity | 0 referencias
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Piso"))
        {
            tocarPiso = true;
        }
    }

}
```

Ilustración 33 Código movimiento jugador

```
public class Enemy_creator : MonoBehaviour
{
    // Start is called before the first frame update
    public GameObject enemigo;
    public float tiempo = 3.0f;
    public Transform[] posicionD;

    void Start()
    {
        StartCoroutine(Generar());
    }

    IEnumerator Generar()
    {
        while (true)
        {
            Cenemigos();
            yield return new WaitForSeconds(tiempo);
        }
    }

    void Cenemigos()
    {
        if (enemigo ==null || posicionD.Length==0)
        {
            return;
        }
        Transform punto = posicionD[Random.Range(0, posicionD.Length)];
        Instantiate(enemigo,punto.position,punto.rotation);
    }

    // Update is called once per frame
}
```

Ilustración 34 Código generación de enemigos

Ilustración 35 Ejecución programa 1

Preexamen

En este repaso de la unidad se practica la creación modificación institución ve a archivos para poder generar contadores dentro de un videojuego en el cual por medio de código se generarán enemigos automáticamente y objetos que sumarán puntos hasta llegar a una meta establecida enviando al usuario a una segunda ventana donde se mostrará los puntos obtenidos junto con su nombre y el protagonista del videojuego

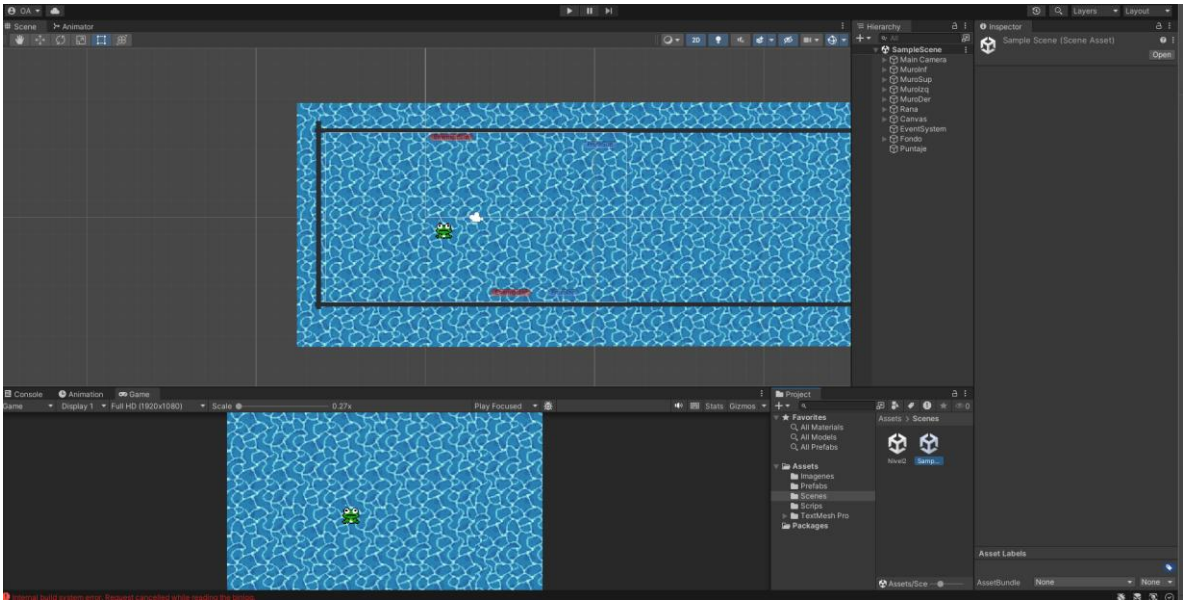

Ilustración 36 Ventana principal preexamen

```
using System.Collections.Generic;
using JetBrains.Annotations;
using UnityEditor;
using UnityEngine;

public class Crearfrutas : MonoBehaviour
{
    // Metodo para crear objetos
    public GameObject objeto;
    public Transform[] posicionD;
    public float tiempo = 3.0f;
    void Start()
    {
        StartCoroutine(Generar());
    }
    IEnumerator Generar()
    {
        while (true) {
            Cobjetos();
            yield return new WaitForSeconds(tiempo);

        }
    }

    // Update is called once per frame
    void Cobjetos()
    {
        if (objeto == null || posicionD.Length == 0)
        {
            return;
        }
        Transform punto = posicionD[Random.Range(0, posicionD.Length)];
        Vector3 pos = punto.position;
        pos.z = 0;  // Fijamos Z en 0
        Instantiate(objeto, pos, punto.rotation);
    }
}
```

Ilustración 37 Código creación de objetos

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Camara : MonoBehaviour
{
    // Codigo para que la camara siga al jugador
    public Transform target;
    public float velocidadCamara = 10.0f;
    public float posInicio;
    public float posFinal;
    float yFija;
    Vector3 fueraCamara;
    Vector3 posicion;

    void Start()
    {
        fueraCamara = transform.position - target.position; //buscar al personaje
        yFija = transform.position.y;
    }

    // Update is called once per frame
    private void LateUpdate()
    {
        float objetivo = target.position.x + fueraCamara.x;
        objetivo = Mathf.Clamp(objetivo, posInicio, posFinal);
        Vector3 objetivoX = new Vector3(objetivo,yFija,transform.position.z);
        transform.position = Vector3.Lerp(transform.position, objetivoX,velocidadCamara*Time.deltaTime);
    }
}
```

Ilustración 38 Codigo camara

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using System.IO;
using UnityEngine.SceneManagement;

public class Contadorenemigos : MonoBehaviour
{
    // Start is called before the first frame update
    public static int contEnemigos = 0;
    public TextMeshProUGUI Puntos;
    public TextMeshProUGUI Nombre;
    public GameObject PanelNombre;
    public TMP_InputField jugador;
    // string dato = "0";
    static StreamReader leer;
    static StreamWriter escribir;

    void Start()
    {

    }
    private void Awake()
    {

    }
    // Update is called once per frame
    void Update()
    {
        Puntos.text = "Puntos: " + contEnemigos.ToString();
        if (contEnemigos == 10)
        {
            Time.timeScale = 0;
            PanelNombre.SetActive(true);

        }
    }
    public static void EnemigosEliminados()
    {
        contEnemigos++;
    }
    public void CrearArchivo()
    {
        if (File.Exists("Puntaje.txt"))
        {
            File.Delete("Puntaje.txt");
        }
        escribir = File.AppendText("Puntaje.txt");
        escribir.WriteLine(contEnemigos + "_" + jugador.text);
        escribir.Close();
        SceneManager.LoadScene("Nivel2");
    }
}
```

Ilustración 39 Codigo contador objetos

```
using System.Collections;
using System.Collections.Generic;
using JetBrains.Annotations;
using UnityEditor;
using UnityEngine;

public class Crearfrutas : MonoBehaviour
{
    // Metodo para crear objetos
    public GameObject objeto;
    public Transform[] posicionD;
    public float tiempo = 3.0f;
    void Start()
    {
        StartCoroutine(Generar());
    }
    IEnumerator Generar()
    {
        while (true) {
            Cobjetos();
            yield return new WaitForSeconds(tiempo);

        }
    }

    // Update is called once per frame
    void Cobjetos()
    {
        if (objeto == null || posicionD.Length == 0)
        {
            return;
        }
        Transform punto = posicionD[Random.Range(0, posicionD.Length)];
        Vector3 pos = punto.position;
        pos.z = 0;  // Fijamos Z en 0
        Instantiate(objeto, pos, punto.rotation);
    }
}
```

Ilustración 40 Creacion de objetos

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Mover : MonoBehaviour
{
    // Codigo para mover al personaje y colisiones
    public float velocidad = 30.0f;
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        if (Input.GetKey(KeyCode.D))
        {
            transform.Translate(Vector2.right*velocidad*Time.deltaTime);
        }
        else if (Input.GetKey(KeyCode.W))
        {
            transform.Translate(Vector2.up * velocidad * Time.deltaTime);
        }
        else if (Input.GetKey(KeyCode.S))
        {
            transform.Translate(Vector2.down * velocidad * Time.deltaTime);
        }
    }
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Enemigo"))
        {
            transform.root.position = Vector3.zero;
            print("Toco");
        }
        if (collision.gameObject.CompareTag("Fruto"))
        {
            Destroy(collision.transform.root.gameObject);
            Contadorenemigos.EnemigosEliminados();
        }
    }
}
```

Ilustración 41 Codigo movimiento jugador

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class MoverEne : MonoBehaviour
{
    // Codigo para  el movimiento de objetos
    public float velodicad = 15.0f;
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        transform.Translate(Vector3.down*velodicad*Time.deltaTime);
    }
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("Muro"))
        {
            Destroy(gameObject);
        }
    }
}
```

Ilustración 42 codigo mover objetos